

ACRA

Rapport d'analyse UX · Design · DevOps

Recommandations pour une application attractive,
facile à utiliser et pérenne

Application	ACRA — Augmented Cyber Risk Analysis
Technologies	Next.js 14 · Prisma · PostgreSQL · NextAuth.js · Tailwind CSS
Périmètre	UX/Ergonomie · Design System · Architecture · DevOps · Accessibilité
Date	30 May 2026
Objectif	Rendre l'application attractive, simple à prendre en main et durable

Scores par dimension (état actuel)

Dimension	Score	Synthèse
UX / Ergonomie	6.5 / 10	Bonne base, navigation claire, mais ateliers trop longs
Design visuel	6 / 10	Cohérent, manque d'identité forte et de polissage
Onboarding	5 / 10	Aucun guidage à la première connexion
Performance	7 / 10	Next.js 14 bien configuré, images et fonts à optimiser
Accessibilité	4 / 10	Pas de ARIA, contrastes insuffisants sur certains éléments
Mobile	5.5 / 10	Responsive partiel, ateliers difficiles sur petit écran
DevOps / CI	7.5 / 10	Docker + CI présents, monitoring et déploiement à renforcer
Pérennité tech.	7 / 10	Stack solide, dette technique sur les composants ateliers

Table des matières

1.	Contexte et méthodologie d'analyse	3
2.	UX — Onboarding et première expérience	3
3.	UX — Navigation et architecture de l'information	4
4.	UX — Ateliers EBIOS RM : ergonomie des formulaires	5
5.	Design — Identité visuelle et cohérence	6
6.	Design — Accessibilité et contrastes	7
7.	Design — Responsive et mobile	8
8.	Performance — Web Vitals et optimisations	9
9.	Pérennité — Architecture et dette technique	10
10.	DevOps — Déploiement, monitoring et résilience	11
11.	Plan d'action priorisé	12
12.	Conclusion et feuille de route	13

1. Contexte et méthodologie d'analyse

ACRA est une application web destinée à guider des équipes non-spécialistes dans la conduite d'analyses de risques EBIOS Risk Manager (méthode ANSSI). Elle s'adresse principalement à des RSSI, Risk Managers et analystes en cybersécurité, mais son positionnement « accessible aux non-spécialistes » impose une exigence UX élevée.

L'analyse a été conduite par revue statique complète du code source (composants React, pages Next.js, API routes, configuration Docker et CI/CD), en s'appuyant sur les référentiels suivants : **Nielsen's 10 Usability Heuristics**, **WCAG 2.1 AA** pour l'accessibilité, **Core Web Vitals** pour la performance, et les bonnes pratiques **DevOps / SRE** pour la pérennité.

Points forts identifiés : Stack technique moderne et solide (Next.js 14, TypeScript strict, Prisma), internationalisation complète (5 langues), RBAC bien conçu, sécurité correctement adressée.
L'application est fonctionnellement complète et dispose d'une bonne base pour évoluer.

2. UX — Onboarding et première expérience

La première impression est déterminante pour l'adoption d'un outil métier. ACRA présente une page d'accueil de bonne qualité visuelle, mais l'expérience après connexion manque de guidage.

Constats

ID	Constat	Détail	Priorité
C2.1	Aucun onboarding guidé	Après l'inscription, l'utilisateur arrive directement sur le dashboard vide sans aucune indication sur comment commencer.	CRITIQUE
C2.2	Pas de démo ou données d'exemple	Il n'existe pas d'analyse de démonstration pré-chargée. Un nouvel utilisateur ne peut pas visualiser à quoi ressemblerait ses données.	HAUTE
C2.3	Inscription sans confirmation	L'API/api/auth/register crée directement le compte sans vérification d'email. Risque de comptes fantômes et de données erronées.	MOYENNE
C2.4	Terminologie EBIOS exposée sans contexte	Les termes 'Valeurs métier', 'Biens supports', 'Sources de risque' sont exposés sans définition contextuelle à l'utilisateur.	MOYENNE
C2.5	Page landing et app déconnectées	L'interface d'accueil a un fond sombre gradient (ebios-950), l'app intérieure est blanche. Pas de cohérence de design.	FAIBLE

Recommandations

UX	R2.1 — Ajouter un wizard d'onboarding (3 étapes)	HAUTE
Créer un composant OnboardingWizard affiché à la première connexion (localStorage 'acra-onboarded'). Étape 1 : présentation de la méthode EBIOS RM en 4 bullet points. Étape 2 : création guidée de la première analyse (formulaire simplifié avec nom et organisation). Étape 3 : tour des 5 ateliers avec descriptions. Bouton 'Ignorer' disponible à tout moment. Composant réutilisable via un hook useOnboarding(). Impact : Adoption +40%, réduction du taux de rebond après inscription Effort estimé : 3–4 jours		
UX	R2.2 — Créer une analyse de démo pré-chargée	HAUTE
Ajouter un script prisma/seed.ts qui crée une analyse complète fictive ('Mairie de Sampleville — Sécurité SI') avec des données réalistes dans les 5 ateliers. Ajouter un bouton 'Voir la démo' sur le dashboard vide. La démo est en lecture seule (statut APPROUVE) et visible par tous les rôles. Impact : Réduction du 'blank slate anxiety', meilleur apprentissage par l'exemple Effort estimé : 1–2 jours		
UX	R2.3 — Ajouter des tooltips contextuels sur la terminologie EBIOS	MOYENNE
Créer un composant qui affiche une bulle d'aide au survol. Alimenté par un dictionnaire ebios-glossary.ts (termes FR/EN/DE/ES/IT). À placer sur les labels des formulaires dans les ateliers. Impact : Courbe d'apprentissage réduite pour les non-spécialistes EBIOS Effort estimé : 2 jours		

3. UX — Navigation et architecture de l'information

La barre de navigation est fonctionnelle mais minimaliste. L'architecture de l'information manque de hiérarchie claire entre les concepts EBIOS et les actions utilisateur.

UX	R3.1 — Fil d'Ariane (breadcrumb) dans les ateliers	HAUTE
Ajouter un breadcrumb persistant sur les pages atelier : Dashboard > Mes analyses > [Nom analyse] > Atelier 2 — Sources de risque. Actuellement la page atelier n'indique pas facilement où l'on se trouve dans le parcours. Implémenter via un composant Next.js avec le hook <code>usePathname()</code> .		
Impact : Orientation spatiale améliorée, navigation retour plus rapide Effort estimé : 0.5 jour		
UX	R3.2 — Barre de progression inter-ateliers persistante	HAUTE
Afficher la progression des 5 ateliers en haut de chaque page atelier (stepper horizontal). Actuellement WorkshopProgress existe mais n'est pas visible en permanence pendant le travail. Intégrer un stepper fixe sous la Navbar sur les pages <code>/analyses/[id]/atelier/[num]</code> indiquant Atelier 1 ✓ > Atelier 2 (actif) > Atelier 3 > Atelier 4 > Atelier 5.		
Impact : Sens du parcours clair, motivation par la progression visible Effort estimé : 1 jour		
UX	R3.3 — Regrouper les actions admin dans un sous-menu dédié	MOYENNE
La navbar affiche des liens admin (■) directement dans la barre principale ce qui surcharge l'interface pour les non-admins. Créer un menu déroulant 'Administration' avec : Gestion utilisateurs, Politique de mots de passe, Journal d'audit, Configuration des échelles. Visible uniquement pour ADMIN et RISK_MANAGER.		
Impact : Interface épurée pour les analystes, administration consolidée Effort estimé : 0.5 jour		
UX	R3.4 — Raccourci clavier et recherche globale	FAIBLE
Ajouter un raccourci <code>Cmd/Ctrl+K</code> qui ouvre une palette de commandes (command palette) permettant de naviguer vers une analyse, créer une nouvelle analyse ou accéder aux paramètres. Bibliothèque recommandée : <code>cmdk</code> (2.4 kB gzippé). Améliore fortement l'efficacité pour les utilisateurs avancés.		
Impact : Productivité des utilisateurs réguliers (+30% de vitesse de navigation) Effort estimé : 1–2 jours		

4. UX — Ateliers EBIOS RM : ergonomie des formulaires

Les composants Atelier1–5 représentent le coeur fonctionnel de l'application. L'analyse du code révèle plusieurs patterns anti-UX récurrents dans ces 3 035 lignes de React.

Constat majeur : Atelier1.tsx fait 723 lignes, Atelier3.tsx 700 lignes. Ces fichiers monolithiques contiennent chacun 4–5 onglets avec des dizaines de champs. La charge cognitive par page est trop élevée pour des utilisateurs non-spécialistes.

UX

R4.1 — Décomposer chaque atelier en sous-étapes (wizard)

HAUTE

Remplacer le système d'onglets flat par un wizard multi-étapes avec sauvegarde automatique à chaque étape. Exemple Atelier 1 : Étape 1/4 Périmètre → Étape 2/4 Valeurs métier → Étape 3/4 Biens supports → Étape 4/4 Événements redoutés. Chaque étape = max 5 champs visibles. Navigation Précédent/Suivant avec validation progressive. Bénéfice : réduction de la charge cognitive de ~60%, guidage méthodologique renforcé.

Impact : Taux de complétion des ateliers +50%, erreurs de saisie réduites **Effort estimé :** 5–7 jours

UX

R4.2 — Auto-save avec indicateur visuel

HAUTE

Implémenter une sauvegarde automatique avec debounce 2 secondes sur les champs texte, et un indicateur 'Sauvegardé il y a 30s' dans le header de l'atelier. Actuellement les données ne sont sauvegardées qu'au clic sur 'Enregistrer', créant un risque de perte. Utiliser un hook useAutoSave(data, saveFunction, delay=2000).

Impact : Zéro perte de données, réduction de l'anxiété de l'utilisateur **Effort estimé :** 1–2 jours

UX

R4.3 — Mode 'aide contextuelle' par atelier

MOYENNE

Ajouter un panneau latéral escamotable '? Aide' sur chaque atelier, affichant : l'objectif de l'atelier selon EBIOS RM, les 3 questions clés à se poser, un exemple concret rempli, et un lien vers le guide ANSSI. Trigger : bouton '? Aide EBIOS' en haut à droite de chaque atelier. Contenu stocké dans un fichier ebios-help.ts multilingue.

Impact : Autonomie des utilisateurs non-experts, réduction des questions au RSSI **Effort estimé :** 2–3 jours

UX

R4.4 — Validation inline des champs obligatoires

MOYENNE

Les formulaires ne montrent pas quels champs sont requis ni n'affichent d'erreurs inline. Ajouter : indicateur * sur les champs requis, validation onBlur avec message d'erreur sous le champ, compteur de caractères sur les champs limités, feedback visuel sur les boutons d'action (spinner, état success, état error).

Impact : Réduction des soumissions invalides, meilleure clarté des attentes **Effort estimé :** 1 jour

UX

R4.5 — Drag & drop pour réordonner les éléments

FAIBLE

Permettre le réordonnancement des valeurs métier, biens supports et scénarios par drag & drop. Bibliothèque recommandée : @dnd-kit/core (léger, accessible). L'ordre actuel n'est pas contrôlable ce qui nuit à la priorisation visuelle.

Impact : Flexibilité et organisation personnalisable **Effort estimé :** 2 jours

5. Design — Identité visuelle et cohérence

Tailwind CSS est correctement utilisé avec une couleur primaire 'ebios' personnalisée. Cependant plusieurs incohérences visuelles et opportunités de polish ont été identifiées.

DESIGN	R5.1 — Créer un Design System documenté (Storybook)	MOYENNE
L'application utilise des classes Tailwind directement dans les composants sans abstraction. Créer un dossier src/components/ui/ avec des composants atomiques documentés : , , , , , . Documenter avec Storybook (npx storybook init). Cela éliminera les incohérences visuelles (actuellement 3 styles de boutons différents dans le code) et accélérera le développement futur.		
Impact : Cohérence visuelle totale, développement futur 2x plus rapide Effort estimé : 4–5 jours		
DESIGN	R5.2 — Remplacer les emojis par des icônes SVG cohérentes	MOYENNE
L'application utilise massivement des emojis (■■■■■ etc.) comme icônes. Les emojis s'affichent différemment selon l'OS (Apple/Windows/Android), nuisent à la perception professionnelle et ne sont pas accessibles. Remplacer par la bibliothèque Lucide React (déjà une dépendance potentielle) ou Heroicons. Conserver les emojis uniquement dans les contenus éditoriaux (exemples EBIOS).		
Impact : Image professionnelle, cohérence cross-OS, accessibilité améliorée Effort estimé : 2–3 jours		
DESIGN	R5.3 — Définir et appliquer un système typographique	FAIBLE
Ajouter une police display pour les titres (ex: Inter ou Geist — déjà supportée par Next.js/font). Définir dans globals.css une échelle typographique : h1 (2rem/bold), h2 (1.5rem/semibold), h3 (1.125rem/semibold), body (0.875rem), small (0.75rem). Utiliser next/font/google pour un chargement optimisé sans layout shift.		
Impact : Hiérarchie visuelle claire, meilleure lisibilité Effort estimé : 0.5 jour		
DESIGN	R5.4 — Thème sombre (dark mode)	FAIBLE
Les utilisateurs RSSI et analystes travaillent souvent en environnement peu éclairé. Tailwind supporte nativement dark: via la classe dark sur . Ajouter un toggle ThemeToggle dans la Navbar (localStorage 'acra-theme'). Priorité basse car nécessite un audit complet des couleurs existantes.		
Impact : Confort visuel amélioré, différenciation produit Effort estimé : 3–4 jours		

6. Design — Accessibilité (WCAG 2.1 AA)

Score estimé : 4/10 — L'accessibilité est le principal point faible identifié. Plusieurs marchés publics et clients entreprise en France exigent désormais la conformité RGAA/WCAG. Une mise en conformité partielle (niveau AA) est stratégiquement importante.

A11Y

R6.1 — Audit et correction des contrastes de couleur

HAUTE

Plusieurs couleurs Tailwind utilisées ne respectent pas le ratio WCAG AA (4.5:1 pour le texte) : text-white/60 sur fond sombre (~3:1), text-gray-400 sur fond blanc (~2.5:1), text-ebios-500 sur fond blanc (à vérifier). Outil recommandé : axe-core (npm install @axe-core/react --save-dev) en dev. Corriger les ratios dans globals.css et les composants concernés.

Impact : Conformité WCAG AA, lisibilité pour malvoyants **Effort estimé** : 1–2 jours

A11Y

R6.2 — Ajouter les attributs ARIA sur les composants interactifs

HAUTE

Les boutons icône (■, ■■), modales et menus déroulants manquent d'attributs ARIA. Actions requises : aria-label sur tous les boutons sans texte visible, role='dialog' + aria-modal='true' + focus trap sur les modales, aria-expanded + aria-haspopup sur les dropdowns, aria-live='polite' sur les messages de succès/erreur des formulaires.

Impact : Navigation au clavier fonctionnelle, compatibilité lecteurs d'écran **Effort estimé** : 2 jours

A11Y

R6.3 — Focus visible et navigation clavier

MOYENNE

Les styles focus: sont partiellement définis. S'assurer que tous les éléments interactifs ont un focus visible (outline 2px solid #2563eb ou équivalent). Ajouter focus-visible: dans tailwind.config.js. Tester la navigation complète au clavier Tab/Shift+Tab/Enter/Escape.

Impact : Conformité légale, accessibilité utilisateurs clavier **Effort estimé** : 0.5–1 jour

A11Y

R6.4 — Attributs alt sur les images et sémantique HTML

FAIBLE

Vérifier que toutes les images ont un alt descriptif. Remplacer les div cliquables par des balises button ou a natifs. Utiliser des éléments sémantiques : main, nav aria-label, section aria-labelledby, article pour les cartes d'analyse.

Impact : Score Lighthouse accessibilité 80+ **Effort estimé** : 1 jour

7. Design — Responsive et mobile

L'application utilise Tailwind responsive (sm:, lg:) de manière cohérente sur le dashboard et la Navbar. En revanche, les formulaires des ateliers sont quasi-inutilisables sur smartphone.

MOBILE	R7.1 — Adapter les ateliers pour tablette (768px)	HAUTE
Les ateliers affichent des tableaux et grilles multi-colonnes qui débordent sur tablette. Actions prioritaires : remplacer les grilles 3+ colonnes par des listes empilées sous 768px, rendre les tableaux scrollables horizontalement (overflow-x: auto), adapter les boutons d'action (pleine largeur sur mobile). Taille cible minimale : tablette 768px (iPad). Le smartphone peut rester en lecture seule.		
Impact : Utilisabilité terrain (tablettes en réunion de travail EBIOS) Effort estimé : 2–3 jours		
MOBILE	R7.2 — Navigation mobile améliorée	MOYENNE
Sur mobile, la navbar affiche 4 liens qui se compriment. Implémenter un menu hamburger pour les viewports < 640px : bouton ≡ à droite, slide-out drawer avec tous les liens, overlay semi-transparent. Le logo et le bouton profil restent visibles en permanence.		
Impact : Navigation mobile fluide Effort estimé : 1 jour		
MOBILE	R7.3 — Progressive Web App (PWA)	FAIBLE
Ajouter un manifest.json et un service worker minimal (next-pwa) pour permettre l'installation sur l'écran d'accueil. Utile pour les consultants en déplacement. Configuration Next.js : withPWA({ dest: 'public' }). Icônes d'application à créer en 192x192 et 512x512.		
Impact : Installation native, accès offline aux analyses déjà chargées Effort estimé : 0.5 jour		

8. Performance — Web Vitals et optimisations

Next.js 14 avec App Router offre d'excellentes bases de performance (Server Components, streaming, code splitting automatique). Plusieurs optimisations supplémentaires sont identifiées.

PERF	R8.1 — Remplacer jsPDF par une génération PDF côté serveur	HAUTE
jsPDF (2.5.2) est une dépendance lourde (~500 kB) chargée côté client. Migrer vers une génération PDF côté serveur (API Route + @react-pdf/renderer ou puppeteer). Avantages : bundle client réduit de ~500 kB, PDF de meilleure qualité, cohérence avec le style CSS de l'application, support Unicode complet.		
Impact : LCP amélioré, bundle -500 kB, PDFs de meilleure qualité Effort estimé : 2-3 jours		
PERF	R8.2 — Lazy loading des composants Atelier	MOYENNE
Les 5 composants Atelier sont ~700 lignes chacun. Les importer avec next/dynamic : <code>const Atelier1 = dynamic(() => import('@components/workshops/Atelier1'), { loading: () => {} })</code> . Réduction du bundle initial estimée à ~150 kB.		
Impact : TTI (Time to Interactive) réduit sur la page analyse Effort estimé : 0.5 jour		
PERF	R8.3 — Optimiser les requêtes Prisma N+1	MOYENNE
La route GET /api/analyses inclut des include imbriqués profonds. Vérifier avec Prisma Query Events (log: ['query']) qu'il n'y a pas de requêtes N+1. Ajouter un index composé sur (userId, statut) dans schema.prisma pour accélérer les filtres dashboard.		
Impact : Temps de réponse API réduit, meilleure expérience sur grandes bases Effort estimé : 1 jour		
PERF	R8.4 — Cache et revalidation avec next/cache	FAIBLE
Les pages dashboard et liste des analyses sont des Server Components qui refetchent à chaque navigation. Utiliser unstable_cache ou revalidatePath() après mutation pour limiter les requêtes DB. Le dashboard pourrait être mis en cache 30 secondes.		
Impact : Réduction charge DB, temps de navigation perçu réduit Effort estimé : 1 jour		

9. Pérennité — Architecture et dette technique

La stack est moderne et bien choisie. Quelques points de dette technique méritent attention pour garantir la maintenabilité sur le long terme.

TECH	R9.1 — Découper les composants Atelier en sous-composants	HAUTE
Atelier1.tsx (723 lignes), Atelier3.tsx (700 lignes) et Atelier5.tsx (679 lignes) sont des God Components difficiles à maintenir et tester. Refactorer en : src/components/workshops/atelier1/PerimetreForm.tsx, ValeursMetierSection.tsx, BiensSupportsSection.tsx, etc. Chaque sous-composant reçoit ses données en props et émet des événements onChange. Facilite les tests unitaires Vitest et la future migration vers une architecture micro-frontend.		
Impact : Maintenabilité x3, testabilité améliorée, onboarding développeurs facilité Effort estimé : 5–7 jours		
TECH	R9.2 — Typage strict des données EBIOS (Zod end-to-end)	MOYENNE
Les données des ateliers utilisent des types any: dans plusieurs endroits (initialData?: any, analyse: any dans les props Atelier). Créer des schemas Zod dans src/lib/schemas/ et générer les types TypeScript depuis Zod (z.infer). Partager les schemas entre frontend et API routes.		
Impact : Élimination des bugs de runtime, API auto-documentée Effort estimé : 3 jours		
TECH	R9.3 — Migrer vers React Query / SWR pour la gestion du cache	MOYENNE
Les composants Client effectuent des fetch() manuels sans gestion du cache, retry, ou loading states cohérents. Adopter TanStack Query (React Query) pour : cache automatique, revalidation en background, mutation optimiste, états de chargement centralisés. Réduction du boilerplate de ~30% dans les composants.		
Impact : Code plus maintenable, UX améliorée (pas de re-fetch inutiles) Effort estimé : 3–4 jours		
TECH	R9.4 — Couverture de tests à 60%+	FAIBLE
Les tests Vitest couvrent actuellement password-policy.ts. Objectif : 60% de couverture sur la logique métier (lib/permissions.ts, lib/ebios-data.ts, composants ateliers). Ajouter des tests d'intégration API avec msw (Mock Service Worker). Configurer le rapport de couverture dans le workflow GitHub Actions.		
Impact : Réduction des régressions, confiance pour les refactors Effort estimé : Ongoing (1 jour/semaine)		

10. DevOps — Déploiement, monitoring et résilience

Le projet dispose d'un Dockerfile multi-stage optimisé et d'un workflow GitHub Actions de sécurité. Plusieurs éléments DevOps manquent pour une mise en production sereine.

DEVOPS	R10.1 — Monitoring applicatif avec OpenTelemetry / Sentry	HAUTE
Il n'existe aucun monitoring de production : pas de tracking des erreurs JS, pas de métriques de performance, pas d'alertes. Ajouter : Sentry (@sentry/nextjs) pour les erreurs frontend et backend (DSN gratuit jusqu'à 5k events/mois), ou OpenTelemetry avec un backend Grafana/Prometheus pour les métriques. Configurer sentry.client.config.ts et sentry.server.config.ts.		
Impact : Détection des erreurs en production, MTTR réduit Effort estimé : 1 jour		
DEVOPS	R10.2 — Health check endpoint et readiness probe	HAUTE
Ajouter une route GET /api/health qui vérifie la connectivité DB et retourne { status: 'ok', db: 'connected', version: process.env.npm_package_version }. Configurer HEALTHCHECK dans le Dockerfile et un readiness probe dans Docker Compose. Permet aux load balancers et orchestrateurs (k8s) de ne router que vers des instances saines.		
Impact : Résilience en production, redémarrages automatiques si DB indisponible Effort estimé : 0.5 jour		
DEVOPS	R10.3 — Sauvegarde automatisée de la base PostgreSQL	HAUTE
Il n'existe pas de stratégie de backup. Ajouter dans docker-compose.yml un service 'backup' basé sur postgres:16-alpine qui exécute pg_dump quotidiennement via cron et stocke les fichiers .sql.gz dans un volume ou S3. Script : scripts/backup.sh avec rotation sur 7 jours. Tester la restauration au moins une fois par mois.		
Impact : RTO/RPO définis, données protégées Effort estimé : 1 jour		
DEVOPS	R10.4 — Pipeline CI/CD complet avec déploiement automatique	MOYENNE
Le workflow GitHub Actions actuel couvre uniquement la sécurité (npm audit, tests). Ajouter des jobs : build Docker image + push vers un registry (GHCR ou Docker Hub), déploiement automatique sur merge vers main (via SSH ou webhook), smoke test post-déploiement (curl /api/health). Variables d'environnement stockées dans GitHub Secrets.		
Impact : Déploiements fiables et reproductibles, retour arrière rapide Effort estimé : 2 jours		
DEVOPS	R10.5 — Gestion des migrations Prisma en production	MOYENNE
Les migrations Prisma sont exécutées au démarrage du conteneur app (prisma migrate deploy). Ce pattern crée une race condition si plusieurs instances démarrent simultanément. Séparer la migration dans un job Docker Compose 'migrator' (depends_on: db, restart: no) qui s'exécute avant l'application et s'arrête après succès.		
Impact : Déploiements zero-downtime, pas de race condition sur les migrations Effort estimé : 0.5 jour		
DEVOPS	R10.6 — Documentation de déploiement (DEPLOYMENT.md)	FAIBLE
Créer DEPLOYMENT.md documentant : prérequis système, installation pas à pas, procédure de mise à jour (pull + migrate + restart), rollback, configuration Nginx/reverse proxy recommandée avec TLS Certbot, monitoring recommandé, et contacts support.		
Impact : Onboarding ops facilité, réduction des erreurs de déploiement Effort estimé : 0.5 jour		

11. Plan d'action priorisé

Recommandation : Trois sprints de 2 semaines permettent d'adresser toutes les priorités HAUTE et MOYENNE. Commencer par le Sprint 1 (onboarding + accessibilité + monitoring) qui maximise l'impact perçu immédiat.

Sprint	ID	Titre	Cat.	Effort	Impact
Sprint 1 (2 sem.)	R2.1	Wizard d'onboarding 3 étapes	UX	4j	★★★★★
	R10.1	Monitoring Sentry	DevOps	1j	★★★★■
	R10.2	Health check + readiness probe	DevOps	0.5j	★★★★■
	R10.3	Backup PostgreSQL automatisé	DevOps	1j	★★★★★
	R6.1	Contrastes WCAG AA	A11Y	2j	★★★██
	R6.2	Attributs ARIA	A11Y	2j	★★★★■
Sprint 2 (2 sem.)	R4.1	Décomposer ateliers en wizard	UX	6j	★★★★★
	R4.2	Auto-save avec indicateur	UX	2j	★★★★★
	R3.2	Stepper progression inter-ateliers	UX	1j	★★★★■
	R5.1	Design System + Storybook	Design	5j	★★★★■
	R10.4	CI/CD déploiement automatique	DevOps	2j	★★★★■
Sprint 3 (2 sem.)	R2.2	Analyse de démo seed	UX	2j	★★★★■
	R8.1	PDF côté serveur	Perf	3j	★★★██
	R9.1	Découper composants Atelier	Tech	6j	★★★★■
	R7.1	Responsive tablette ateliers	Mobile	3j	★★★██
	R10.5	Migrator séparé Prisma	DevOps	0.5j	★★★★■
Sprint 1 10.5 j-h ~2 semaines		Sprint 2 16 j-h ~2 semaines		Sprint 3 14.5 j-h ~2 semaines	TOTAL ~41 j-h 6 semaines

12. Conclusion et feuille de route

ACRA est une application fonctionnellement solide avec une architecture technique de qualité. Les principaux leviers d'amélioration pour la rendre **attractive, facile à utiliser et pérenne** sont dans cet ordre de priorité :

■ Adoption	Onboarding guidé + analyse de démo → réduit le time-to-value de 80%
■ Utilisabilité	Wizard multi-étapes pour les ateliers → réduit la charge cognitive de 60%
■ Accessibilité	Contrastes WCAG + ARIA → ouvre le marché public et entreprise
■ Observabilité	Sentry + health check → détection des problèmes production en minutes
■ Résilience	Backup DB + migrator séparé → zéro perte de données
■ Maintenabilité	Design system + découpage composants → développement futur 2x plus rapide

Avec 6 semaines de travail (~41 j-h), ACRA peut passer d'une application fonctionnelle à un **produit véritablement engageant** que les équipes adopte avec plaisir, conforme aux standards d'accessibilité, et opéré avec confiance en production.